

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Цветкова Татьяна

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург 2026 г.

Задача

Тема: Лабораторная работа №1. Реализация REST API на основе boilerplate.

Вариант: «Платформа для фитнес-тренировок и здоровья».

Цель работы: на основе выбранного boilerplate реализовать полностью работающее REST API в соответствии со схемой БД (Д31) и OpenAPI-описанием эндпоинтов (Д32). В рамках работы необходимо реализовать модели, представления и контроллеры.

Задание:

1. Использовать подходящий boilerplate для реализации REST API.
2. Реализовать модели (entities) согласно ER-диаграмме из Д31.
3. Реализовать представления и контроллеры под все эндпоинты, описанные в OpenAPI-схеме (Д32).
4. Продемонстрировать полностью работающее API в соответствии с вариантом: вход, регистрация, личный кабинет (трекинг прогресса, планы тренировок), поиск тренировок с фильтрацией по уровню, типу и продолжительности, страница тренировки с видео, описанием и инструкциями, блог о здоровье и питании.

Ход работы

В качестве шаблона выбран boilerplate `express-typeorm-boilerplate`, адаптированный под задачу. Итоговая структура построена по слоистой архитектуре: `Routes` → `Controllors` → `Services` → `Repositories (TypeORM)` → `DB`. Валидация входных данных вынесена в DTO с использованием `class-validator`, обработка ошибок — в общий `middleware`.

1. Стек технологий

- Node.js 20+ / TypeScript 5;
- Express 4 — HTTP-сервер и маршрутизация;
- TypeORM 0.3 — ORM, поддержка SQLite (по умолчанию) и PostgreSQL;
- jsonwebtoken и bcrypt — аутентификация и хеширование паролей;
- class-validator / class-transformer — валидация DTO;

- `swagger-ui-express` + `yamljs` — интерактивная документация (OpenAPI 3.0);
- `helmet`, `cors`, `morgan` — безопасность и логирование.

2. Структура проекта

Каталог `src/` разделён на подкаталоги `config`, `entities`, `dto`, `services`, `controllers`, `routes`, `middlewares`, `utils`, `seeds`. Точка входа — `server.ts`, сборка приложения — `app.ts`, описание подключения к БД — `config/data-source.ts`. OpenAPI-спецификация хранится в корне (`openapi.yaml`) и отдаётся через Swagger UI по адресу `/api/docs`.

3. Реализация моделей (entities)

В соответствии с ER-диаграммой из ДЗ1 описаны девять сущностей TypeORM с UUID-ключами: `User` (роль `user/admin/trainer`, антропометрия, уровень подготовки), `WorkoutCategory`, `Workout` (видео, инструкция, тип, уровень, длительность, оборудование, мышцы), `WorkoutPlan`, `PlanItem` (элемент плана: тренировка + день + статус выполнения), `ProgressEntry` (запись о выполненной тренировке), `BlogCategory`, `BlogPost` (`slug`, теги, флаг публикации) и `BlogComment`. Поле `passwordHash` помечено как `select: false` — пароль не попадает в обычные выборки. Удаление сущностей-родителей описано через `CASCADE/SET NULL` в зависимости от смысла связи.

4. DTO, валидация и обработка ошибок

Каждый ресурс имеет DTO для создания, обновления и фильтрации. Валидация выполняется функцией `validateDto`, которая приводит `payload` к классу через `class-transformer` и проверяет через `class-validator`. При обнаружении ошибок выбрасывается `ValidationError` (422) со списком нарушенных правил по полям. Введена иерархия `AppError`: `NotFoundError` (404), `UnauthorizedError` (401), `ForbiddenError` (403), `ConflictError` (409), `ValidationError` (422). Глобальный `errorHandler` преобразует их в JSON-ответ единого формата.

5. Сервисы (бизнес-логика)

Каждой группе ресурсов соответствует свой сервис: `AuthService`, `UserService`, `WorkoutService`, `WorkoutPlanService`, `ProgressService`, `BlogService`. В `WorkoutService.list` использован `QueryBuilder` с фильтрацией по `type`, `level`, `minDuration`, `maxDuration`, `categoryId` и поиском по подстроке,

а также пагинацией. `WorkoutPlanService` проверяет, что план принадлежит текущему пользователю (в противном случае выбрасывается `ForbiddenError`). `ProgressService.stats` возвращает агрегаты: суммарную длительность, количество калорий, средний рейтинг.

6. Контроллеры и представления

Контроллеры (`AuthController`, `UserController`, `WorkoutController`, `WorkoutPlanController`, `ProgressController`, `BlogController`) реализованы как тонкий слой: они валидируют входные данные, вызывают сервис и формируют HTTP-ответ. В качестве «представлений» REST-приложения выступают `express`-роутеры (`routes/*.routes.ts`) и сгенерированный Swagger UI: интерактивная документация по адресу `/api/docs` позволяет просматривать и тестировать все эндпоинты.

7. Аутентификация и авторизация

Регистрация хеширует пароль `bcrypt` (10 раундов). При логине формируются два JWT: `accessToken` (1 день) и `refreshToken` (7 дней). `Middleware authenticate` проверяет Bearer-токен из заголовка `Authorization` и кладёт декодированный `payload` в `req.user`. `Middleware authorize(...roles)` ограничивает доступ по ролям: операции изменения тренировок и публикаций блога разрешены только пользователям с ролью `admin` или `trainer`.

8. Реализованные эндпоинты

- `/api/auth`: `register`, `login`, `refresh` — регистрация и аутентификация;
- `/api/users/me` (`GET/PATCH/DELETE`) — личный кабинет;
- `/api/workouts` — поиск тренировок с фильтрами `type`, `level`, `minDuration`, `maxDuration`, `categoryId`, `search`; `GET /workouts/{id}` — страница тренировки с видео, описанием и инструкциями; `CRUD` и категории;
- `/api/workout-plans` — личные планы тренировок и управление элементами плана, отметка выполнения;
- `/api/progress` — трекинг прогресса с фильтрами по периоду и сводная статистика `/progress/stats`;
- `/api/blog` — блог о здоровье и питании: статьи (получение по `UUID` или `slug`), категории, комментарии.

9. Запуск и проверка

Подготовлен seed-скрипт (`npm run seed`), который создаёт администратора (`admin@fitness.local / admin12345`), демо-пользователя (`user@fitness.local / user12345`), категории тренировок и блога, а также пять примеров тренировок и две статьи. Запуск в режиме разработки: `npm run dev` — сервер поднимается по адресу `http://localhost:3000/api`, Swagger UI доступен на `/api/docs`. Работоспособность проверена сценариями: регистрация → логин → получение `accessToken` → запрос `/users/me` → поиск тренировок с фильтрами → создание плана → отметка выполнения элементов плана → запись в прогресс → получение статистики → создание поста блога (`admin`) → добавление комментария.

Вывод

В ходе выполнения лабораторной работы на основе `boilerplate Express + TypeORM + TypeScript` разработано полноценное REST API для платформы фитнес-тренировок и здоровья. Реализованы модели (девять сущностей `TypeORM`), представления (`express`-роутеры + интерактивная Swagger-документация) и контроллеры — в соответствии с разделением на слои `Routes / Controllers / Services`.

Все требования варианта закрыты: реализованы регистрация и вход (JWT, `access + refresh`), личный кабинет с обновлением профиля, поиск тренировок с фильтрацией по уровню, типу и продолжительности, страница тренировки с видео, описанием и инструкциями, личные планы тренировок с отметкой выполнения, трекинг прогресса и агрегированная статистика, а также блог о здоровье и питании с комментариями. Введена ролевая модель (`user, trainer, admin`) — операции изменения тренировок и постов доступны только администраторам и тренерам.

Применены практики защиты и качества кода: валидация всех DTO через `class-validator`, единая иерархия `AppError` и общий `error-handler`, хеширование паролей `bcrypt`, защита заголовков `helmet`, ограничение CORS, логирование `morgan`. Подготовлена OpenAPI-спецификация и сид-скрипт для быстрого старта.

Цель лабораторной работы достигнута: REST API спроектировано в соответствии с ДЗ1 и ДЗ2, полностью реализовано и проверено

работоспособностью базовых сценариев. Решение легко расширяется (миграции, тесты, интеграция S3 для загрузки видео, кеширование).